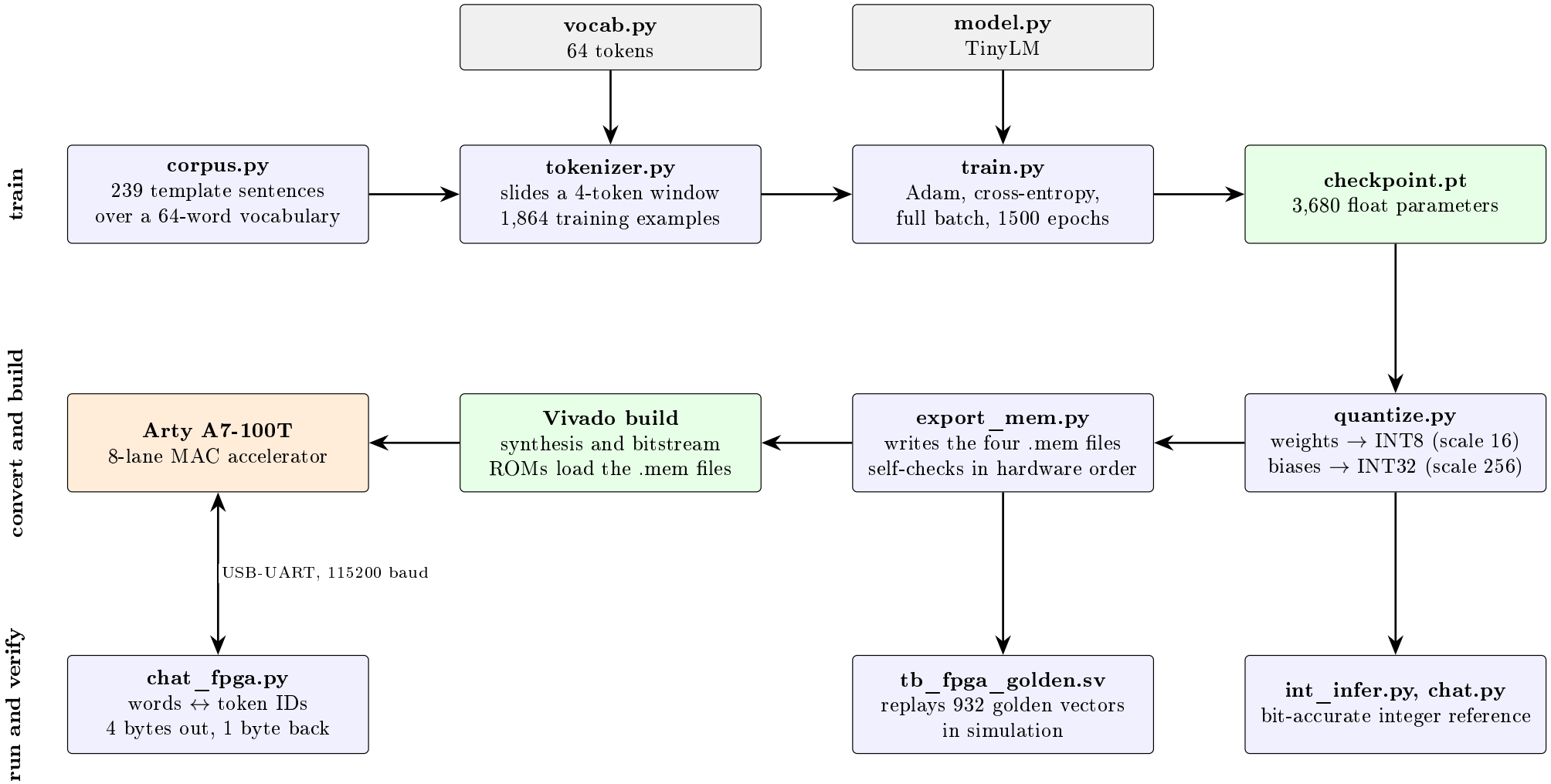
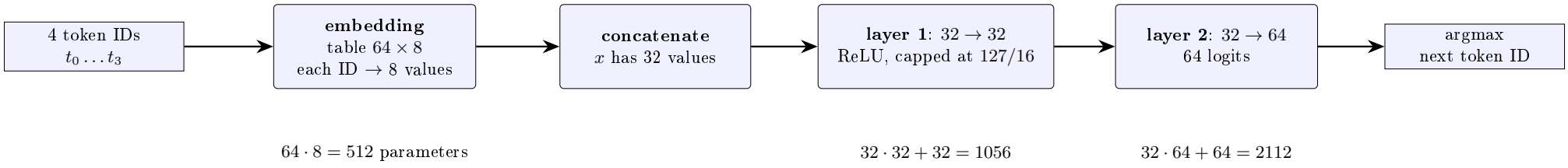
1. Language model: from sentences to the FPGA



The only file the hardware side depends on is `checkpoint.pt`. Everything after it is deterministic: quantizing, writing the memory files, and checking them against the integer reference. If the model is retrained, one command (`python -m langModel.export_mem`) regenerates all four files and re-verifies them.

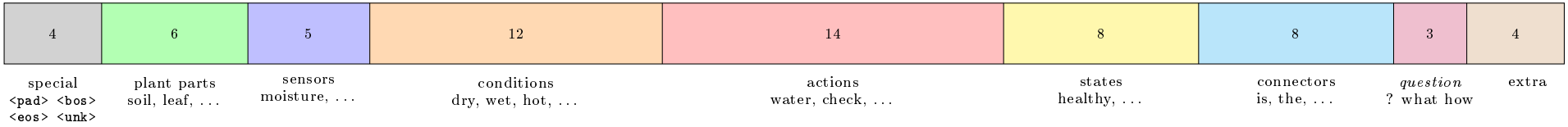
2. The model



total $512 + 1056 + 2112 = 3680$ **parameters**

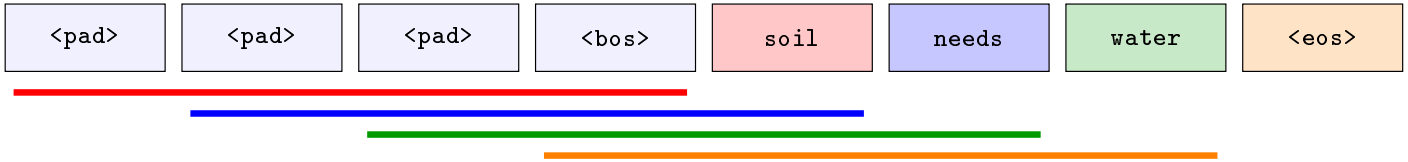
The cap on the hidden layer is the one thing that is not an ordinary ReLU. The hardware stores hidden activations as INT8 and clamps them to $[0, 127]$ at scale 16, which is a real value of $127/16 \approx 7.94$. Training uses the same cap (`HIDDEN_MAX` in `model.py`), so the float model never relies on values the hardware cannot represent. It matters: about 10% of hidden activations reach the cap.

the fixed 64-word vocabulary (the segment widths are proportional to the number of words)



3. Training examples and the generation loop

A sentence from the corpus: “soil needs water”. The tokenizer pads and wraps it, then every 4-token window predicts the token after it.



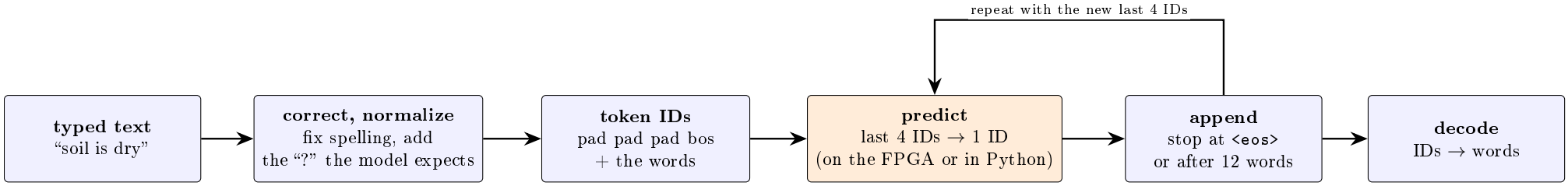
context (pad, pad, pad, bos) → target `soil`

context (pad, pad, bos, soil) → target `needs`

context (pad, bos, soil, needs) → target `water`

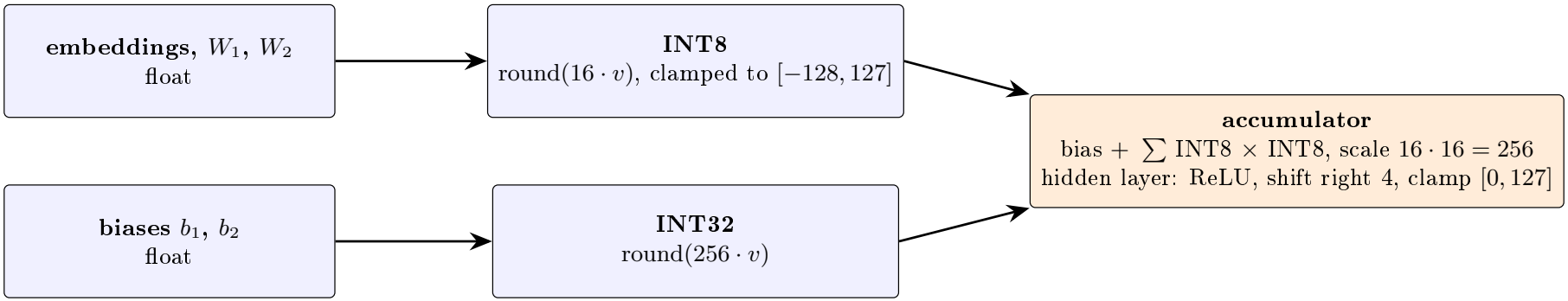
context (bos, soil, needs, water) → target `<eos>`

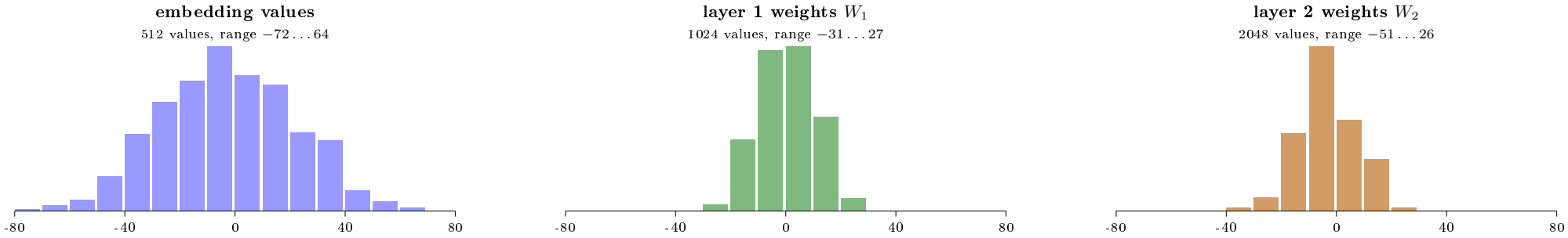
All 239 sentences give
1,864 training examples,
of which **632** have a distinct context.



The model only ever predicts one token. Autoregressive generation is the loop around it, which is why the hardware needs no tokenizer: the host sends 4 token IDs, receives 1, and does the rest.

4. Quantization and what the numbers look like





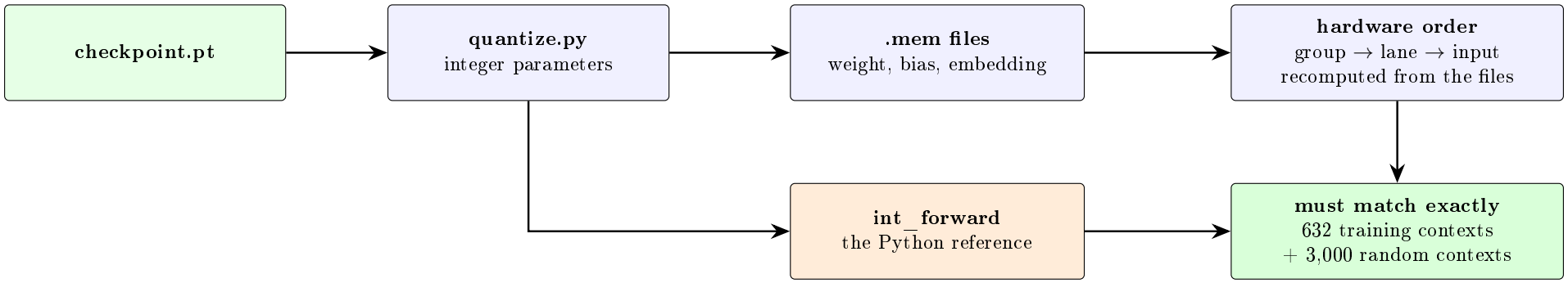
INT8 value (histograms of the quantized values in `checkpoint.pt`); nothing is clipped, so the scale of 16 fits this model with room to spare

Measured on `checkpoint.pt`	**Result**
Parameters, shapes	3,680, all match the shared spec
INT8 values that clip	0 of 3,584
Largest accumulator magnitude over all 632 contexts	20,262 (INT32 would overflow at 2,147,483,647)
Hidden activations: zero / between / at the cap 127	10,102 (50.0%) / 8,139 (40.2%) / 1,983 (9.8%)
Accuracy on the training examples, float / integer	75.3% / 75.2%
Float and integer argmax agree	94.8% of examples

The two models predict the same accuracy even though 5% of individual argmax choices differ, so quantization is not what limits accuracy. Many contexts in the corpus legitimately have several valid next words, which caps what any model can score on them.

5. Files and how the export is checked

File	Role
`vocab.py`	the 64-word vocabulary and the ids of the special tokens
`corpus.py`	239 sentences built from a few templates, including condition-to-advice tables
`tokenizer.py`	word ↔ id, and the sliding 4-token window that makes (context, target) pairs
`model.py`	TinyLM: embedding, two linear layers, hidden activations capped at 127/16
`train.py`	full-batch Adam training, saves `checkpoint.pt`, prints sample generations
`quantize.py`	float → INT8 (scale 16) and INT32 (scale 256)
`int_infer.py`	integer-only forward pass and generation loop: the reference the hardware must match exactly
`export_mem.py`	reads the checkpoint (no PyTorch needed), writes the four `mem/` files, then verifies them
`chat.py`	text prompt loop on the integer reference (spelling correction, phrase normalizing)
`chat_fpga.py`	the same loop with every prediction sent to the board over UART



This check runs every time the files are exported. It reads the written `.mem` files back, rebuilds the matrices exactly the way the hardware addresses them, and requires every one of the 64 logits to equal `int_forward`'s. A mistake in the byte order or the address layout would fail here, before any Verilog is involved. The same export writes `golden_vectors.mem`, which the simulation testbench replays through the real design.